IronFence Inteligência Artificial

Teste FullStack Sênior - Laravel/PHP

> 💡 ***Uma Grande Oportunidade!***
> *Você está se candidatando para compor um time de excelência em uma empresa líder em seu segmento. Este teste representa uma excelente oportunidade para candidatos preparados demonstrarem suas habilidades técnicas e serem reconhecidos por sua expertise. É sua chance de brilhar e mostrar do que é capaz!*

Objetivo

Desenvolver uma aplicação avançada de gerenciamento de tarefas usando Laravel/PHP com funcionalidades de tempo real, cache e processamento assíncrono, demonstrando expertise em desenvolvimento sênior.

Esforço estimado: 4 dias de trabalho **Prazo:** 1 semana

Projeto

Sistema de tarefas com funcionalidades avançadas onde usuários podem gerenciar tarefas com notificações em tempo real, performance otimizada e processamento em background. Deve incluir dashboard responsivo, suporte multilíngue e arquitetura escalável.

Entrega: O projeto deve ser compartilhado em repositório **privado** do GitHub com acesso para [e-mail removido]

Tecnologias (escolha uma opção de cada)

Frontend (escolha uma opção)

Opção 1: Laravel Livewire + Bootstrap - Livewire para componentes interativos - Bootstrap para estilização e responsividade - Integração nativa com WebSocket - Alpine.js pode ser utilizado caso considere necessário

Opção 2: Bootstrap + Blade + React.js (opcional) - Bootstrap puro inicializado via templates Blade - Foco no uso tradicional do Bootstrap com Blade templates - React.js é completamente opcional para componentes específicos que realmente necessitem maior interatividade - JavaScript/jQuery para interações básicas quando React não for usado

Banco de Dados

- **MySQL/MariaDB:** banco relacional tradicional
- **SQLite:** banco de arquivo local

Infraestrutura Adicional

- **Sistema de Filas:** Qualquer driver do Laravel (database, redis, sync, etc.)
- **WebSocket:** Pusher ou Laravel WebSockets

Requisitos Básicos

Autenticação

- Sistema de registro e login
- Proteção de rotas para usuários autenticados
- Middleware de autenticação

Tarefas (CRUD)

Cada tarefa deve ter: - Título - Descrição
- Data de vencimento - Status (pendente, em progresso, concluída) - Prioridade (baixa, média, alta) - Associação ao usuário criador

Interface Responsiva

Opção 1 - Livewire + Bootstrap: - Componentes Livewire reativos - Bootstrap para responsividade - Atualizações em tempo real automáticas

Opção 2 - Bootstrap + Blade + React (opcional): - Bootstrap para design responsivo - Templates Blade organizados - JavaScript para WebSocket client - React opcional para componentes complexos

Multilíngue

- Suporte a Português e Inglês
- Alternância entre idiomas
- Tradução completa da interface

Funcionalidades Sênior

1. WebSocket - Tempo Real

Implementação obrigatória: - Notificações em tempo real quando as próprias tarefas do usuário são criadas/atualizadas - Dashboard do usuário atualiza automaticamente suas tarefas - Eventos direcionados apenas para o usuário proprietário das tarefas

Teste de funcionalidade: - Abrir a aplicação em múltiplos navegadores ou dispositivos com o mesmo usuário - Criar/editar uma tarefa em um navegador - A mudança deve aparecer **instantaneamente** nas outras instâncias do mesmo usuário - Dashboard deve atualizar contadores em tempo real - Notificações visuais devem aparecer nas outras telas do usuário

Tecnologia: livre escolha (Laravel Broadcasting com Reverb, Pusher Channels, Laravel WebSockets, Socket.io, etc.)

2. Sistema de Filas

Implementação obrigatória: - Sistema de filas do Laravel (driver livre: database, redis, sync, etc.) - Envio de emails de notificação em background - Emails disparados quando: - Nova tarefa é criada - Tarefa é atribuída a outro usuário - Tarefa tem prazo próximo ao vencimento

Dashboard de Gerenciamento de Filas (obrigatório): - Deve entregar uma interface para monitorar as filas - Pode ser Laravel Horizon, custom dashboard, ou outra solução - Deve mostrar: jobs processados, failed jobs, workers ativos - Interface acessível via web

Funcionalidades esperadas: - Queue workers configurados - Jobs com retry e timeout - Failed jobs handling - Dashboard funcional para monitoramento

3. Serviço de Alerta

Implementação obrigatória: - Serviço que monitora tarefas próximas ao vencimento - Dispara alerta 30 minutos antes da tarefa vencer - Pode ser implementado via: - Scheduled jobs (Laravel Scheduler) - Queue jobs com delay - Cron jobs customizados - Alerta pode ser: email, notificação WebSocket, ou ambos

Funcionalidades esperadas: - Verificação automática de prazos - Disparo preciso do alerta (30min antes) - Evitar alertas duplicados - Log dos alertas enviados

API RESTful

- **Opção 1 (Livewire + Bootstrap):** API opcional para WebSocket
- **Opção 2 (Bootstrap + Blade + React):** API necessária para React e WebSocket
- Endpoints protegidos por autenticação
- Rate limiting básico (opcional)

Dashboard Avançado

- Contadores de tarefas por status
- Gráficos simples de produtividade

- Atualizações em tempo real via WebSocket
- Filtros por período e prioridade

Docker

- Dockerfile para Laravel
- docker-compose.yml com:
- Servidor web
- PHP
- Banco de dados escolhido
- Redis (opcional, se escolher como driver de filas)
- Queue worker
- Dashboard de filas (se aplicável)
- Execução com `docker-compose up`

Entregáveis

- **Repositório GitHub privado** compartilhado com [e-mail removido]
- Código fonte completo
- Arquivos Docker
- README detalhado com:
- Instruções de instalação
- Configuração do sistema de filas escolhido
- Configuração da tecnologia WebSocket escolhida
- Como executar queue workers
- Como acessar dashboard de filas
- Como funciona o serviço de alerta
- Tecnologias escolhidas e justificativa
- Arquitetura da solução

Avaliação Sênior

Funcionalidade (20%)

- Todos os requisitos básicos + funcionalidades sênior
- WebSocket funcionando corretamente para tarefas do próprio usuário
- Sistema de filas processando emails adequadamente
- Serviço de alerta funcionando nos prazos corretos
- Dashboard de filas acessível e funcional

Arquitetura e Código (25%)

- Código limpo e bem estruturado
- Padrão MVC respeitado
- Separação adequada de responsabilidades
- Tratamento de erros robusto

Performance (20%)

- Queries otimizadas
- Jobs não bloqueantes
- WebSocket com boa performance
- Configuração adequada de workers

Infraestrutura (20%)

- Docker configurado corretamente
- Sistema de filas funcionando adequadamente
- Queue workers estáveis
- Dashboard de filas configurado e acessível
- Serviço de alerta configurado adequadamente
- Configuração de produção adequada

Documentação (15%)

- README completo e claro
- Decisões arquiteturais documentadas
- Instruções de deploy
- Troubleshooting básico

Distribuição de Tempo Sugerida

Dia 1 - CRUD (8h): - Setup do projeto e Docker (1h) - Autenticação e middleware (1.5h) - Models, migrations e relacionamentos (1.5h) - CRUD completo de tarefas (3h) - Interface básica e validações (1h)

Dia 2 - Filas (8h): - Configuração do sistema de filas escolhido (1h) - Implementação de jobs de email (2h) - Queue workers e configuração (1h) - Dashboard de gerenciamento de filas (3h) - Testes e debugging de filas (1h)

Dia 3 - WebSocket (8h): - Setup da tecnologia WebSocket escolhida (2h) - Implementação de broadcasting (2h) - Frontend para receber eventos (2h) - Dashboard em tempo real (1.5h) - Testes multi-browser (0.5h)

Dia 4 - Serviço de Alerta (8h): - Implementação do serviço de alerta (3h) - Integração com filas/WebSocket (2h) - Multilíngue e ajustes finais (2h) - Documentação completa (1h)

Dicas Sênior

WebSocket: - Escolha a tecnologia que domina melhor - Implemente reconnection automática no frontend - Considere rate limiting para eventos - Teste em múltiplos navegadores

Sistema de Filas: - Escolha o driver que domina melhor (database, redis, sync) - Use unique jobs quando necessário para emails - Implemente logging para debugging de jobs - Configure retry attempts adequados para emails - Dashboard pode ser custom ou usar Horizon

Serviço de Alerta: - Use Laravel Scheduler para verificações periódicas - Considere timezone dos usuários - Evite alertas duplicados com flags ou cache - Integre com sistema de filas para performance

Geral: - Foque na arquitetura antes da implementação - Use environment variables para configurações - Implemente graceful degradation - Documente decisões técnicas importantes

Processo de Avaliação

Critério de Funcionalidade

- O projeto **DEVE** rodar completamente com `docker-compose up` ou script shell simples
- Não devem ser necessárias configurações manuais complexas
- Todas as funcionalidades devem estar operacionais após a execução

Feedback e Próximos Passos

- **Candidatos aprovados:** todos que entregarem projeto funcionando em Docker Compose receberão **call de feedback**
- **Candidatos eliminados:** projetos que não rodarem com `docker-compose up` ou script shell **poderão ser eliminados** sem call de feedback

Expectativas de Entrega

- Código funcional e testado
- Documentação clara no README
- Configuração Docker completa e funcional
- Projeto deve "rodar de primeira" seguindo as instruções

Observações

- Uso de IA (ChatGPT) é permitido e bem-visto

- Demonstre conhecimento de padrões e boas práticas
- Priorize funcionalidade sobre interface elaborada
- Documente decisões técnicas importantes

Boa sorte! Esperamos ver uma solução robusta e bem arquitetada.

```
<?= "Que a força do código esteja com você! Iniciando em " . date('Y-m-d H:i:s') . " ⚡" ?>
```